Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Asıl anlatım yaklaşık 00:04'te başlar.
- Kaynak: Mano & Kime'nin kitabı. G/Ç bölümü ve önbellek için 14. bölüm.
- Pipeline hakkında kısa cevap
  - Veri bağımlılığı derleyici (compiler) düzeyinde yazılımla ya da donanımla çözülebilir. Ayrıntısı Bilgisayar Mimarisi dersinde.
- G/Ç tekrarı
  - Çevre birimleri, G/Ç arayüzü, isolated I/O ve memory-mapped I/O, chip select, strobing ve handshaking.
- Seri iletişim
  - Asenkron ve senkron seri haberleşme. Start, stop ve parity bitleri. UART/USART.
- Veri transfer yöntemleri (modes of transfer)
  - Program kontrollü transfer (programmed I/O, yoklama/polling).
  - Kesme ile başlatılan transfer (interrupt-initiated).
  - Doğrudan bellek erişimi (DMA, direct memory access).
  - Dördüncü seçenek: G/Ç işlemcisi (I/O processor). Mainframe'lerde kullanılır.
- Öncelikli kesme donanımı
  - Daisy chain (seri) öncelik ve paralel öncelik donanımı.
- DMA denetleyicisi (DMA controller)
- Örnek problemler
  - Arayüzü 240-243 adreslerine yerleştirme, doğrusal seçim ile kaç arayüz bağlanabileceği, seri iletim süresi.
- Önbelleğe giriş (cache memory)
  - SRAM/DRAM, hit/miss, direct mapped, fully associative, set associative önbellek, blok (line) kavramı.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu konular mesleğin "arka yüzüdür". Bilgisayarın portlarla ve cihazlarla nasıl haberleştiğini teknik olarak öğreniyorsunuz.
- Program kontrollü transfer işlemciyi bloke eder.
  - Bilgisayarda kullanılmaz. Tek işi olan küçük, özel (dedicated) bir donanımda kullanılabilir.
  - Örnek: ortam karardığında ışığı açan bir devre.
- Bilgisayarda tercih edilen yöntem kesme ile transferdir. Büyük veri blokları için DMA kullanılır.
- Daisy chain'de acknowledge işaretini ilk alan birim en yüksek önceliğe sahiptir.
- DMA'da CPU devre dışı bırakılır.
  - Bus request ve bus grant işaretleri bu yüzden çok önemlidir.
- Önbellek sorularının hepsi matematiktir. Mantığı anlarsanız kolayca çözersiniz.
- Önbellek kapasitesi söylenirken yalnızca veri alanı sayılır. Tag alanı hesaba katılmaz.
- Kitabın bölüm sonu sorularını çözün.
  - Ezber gerekmez. Soruları dikkatli okuyun, mantıkla çözün.

Kısa Tekrar Notları

- Isolated I/O (8086): bellek ve G/Ç uzayı ayrı. M/$\overline{IO}$ kontrol ucu seçer. IN/OUT komutları kullanılır.
- Memory-mapped I/O: tek uzay, G/Ç'ye de MOV ile erişilir.
- k iç register'lı arayüz G/Ç uzayında arka arkaya k adres kaplar.
- Asenkron çerçeve: start biti (0) + 7/8 veri biti (D0 önce) + parity (isteğe bağlı) + 1, 1.5 ya da 2 stop biti (1).
- 8 veri + 1 start + 2 stop = 11 bit. Parity eklenirse 12 bit.
- UART yalnızca asenkron. USART senkron veya asenkron.
- Daisy chain tablosu: PI=1 ve RF=1 ise Enable=1, PO=0. PI=1 ve RF=0 ise PO=1. PI=0 ise PO=0, Enable=0.
- Paralel öncelik: interrupt register + mask register + 4'ten 2'ye öncelik kodlayıcı (priority encoder). V çıkışı CPU'ya kesme isteğidir.
- DMA denetleyicisi: adres register'ı (her transferde +1), word count register'ı (her transferde −1), control register.
- DMA adımları: DMA request → CPU register'ları yükler → bus request → bus grant → DMA acknowledge → transfer.
- Direct mapped (10 bit adres, 8 satır, 4 byte/satır): tag 5 | index 3 | byte offset 2.
- Fully associative: tag 8 | byte offset 2. Satır sayısı kadar karşılaştırıcı gerekir.
- 2-way set associative (8 satır = 4 küme × 2 yol): tag 6 | index 2 | byte offset 2. 2 karşılaştırıcı gerekir.
- Eşleştirme (match logic): her tag biti için XOR, çıkışları NOR. NOR = 1 → eşleşme.

Detaylı Açıklamalar

1. G/Ç tekrarı

- Klavye: satırları süren decoder, sütunları okuyan MUX. 4×4 tuş takımı örneği Mikroişlemciler dersinin konusudur.
- Her işlemcinin arayüz entegreleri vardır. Örnek: 8086 için paralel port arayüzü 8255.
  - Basit birimler (ör. ADC entegresi) arayüzsüz de bağlanabilir.
- **Isolated I/O:** 8086'da bellek ve G/Ç uzayı ayrıdır. M/$\overline{IO}$ ucu 1 ise belleğe, 0 ise G/Ç uzayına erişilir.
- **Memory-mapped I/O:** aynı adres uzayının bir bölgesi belleğe, bir bölgesi G/Ç'ye ayrılır. Aynı komutlarla erişilir.
- Arayüzü belli bir adres aralığına yerleştirmek için adres yolu bir adres çözücüden (address decoder) geçirilip chip select üretilir.
- Arayüz ile cihaz arasında strobing (tek tel, yetersiz) ya da handshaking (request/reply, güvenli) kullanılır.

2. Paralel ve seri iletişim

- **Paralel:** n bit aynı anda n telden. Hızlı ama çok tel ister. Kısa mesafede kullanılır.
  - Paralel portların yerini büyük ölçüde hızlı USB portları aldı.
- **Seri:** tek tel, bitler belli bir bit hızıyla (bit rate) arka arkaya gider. Daha yavaştır.
- **Asenkron seri haberleşme:**
  - Hat boşta (idle) iken lojik 1'dir.
  - Karakter start bitiyle (her zaman lojik 0) başlar.
  - Sonra D0'dan (en düşük anlamlı bit) D7'ye veri bitleri gelir. 7 bitlik ASCII kullanan sistemler de vardır.
  - Varsa hata kontrolü için parity biti eklenir.
  - Stop biti (her zaman lojik 1) karakterin bittiğini gösterir. 1, 1.5 ya da 2 bit olabilir.
- **Senkron seri haberleşme:** veri çerçeveler (frame) hâlinde gönderilir. Çerçeve senkronizasyon karakterleri ve STX (start of text) gibi ASCII kodlarıyla başlar.
- 8251 USART Mikroişlemciler dersinin konusudur.

3. Program kontrollü transfer (programmed I/O)

- Yapı: CPU + arayüz + giriş birimi. Arayüzde data register ve status register vardır.

  - Örnek adresler: data register A8h, status register A9h.
- Adımlar:
  1. Cihaz veriyi I/O yoluna koyar ve ready işaretini aktif yapar.
  2. Arayüz veriyi data register'a yazar ve acknowledge işaretini aktif yapar.
  3. Status register'daki bayrak biti (F, en düşük anlamlı bit) 1 olur: "data register dolu".
- CPU döngü içinde bayrağı yoklar (polling):

```
KONTROL: IN   AL, 0A9h    ; status register'ı oku
         TEST AL, 01h     ; bayrak biti 1 mi?
         JZ   KONTROL     ; 0 ise tekrar kontrol et
         IN   AL, 0A8h    ; data register'ı oku
         ; veriyi belleğe yaz, transfer bitmediyse başa dön
```

- Sorun: CPU sürekli bu döngüde kalır. Klavyede iki tuşa basana kadar döngü binlerce kez döner. Verimli değildir.

4. Kesme ile başlatılan transfer

- Data register'a veri gelince bayrak 1 olur ve CPU'ya kesme isteği (interrupt request) gider.
- CPU yürüttüğü komutu bitirir ve kesme alt programına (interrupt service routine) dallanır.
- Alt programa arayüzü okuyan komutlar yazılır.
- Birden fazla kaynak olduğu için öncelik lojiği gerekir.

5. Daisy chain öncelik donanımı

- 3 cihazın kesme istekleri bir OR kapısıyla toplanıp CPU'ya gider.
- CPU komutu bitirince interrupt acknowledge gönderir.
- Acknowledge cihazlara sırayla (PI → PO) aktarılır:
  - En soldaki (Device 0) en yüksek, en sağdaki en düşük önceliklidir.
  - İsteği olan ilk cihaz vektör adresini (VAD) veri yoluna bırakır ve acknowledge'ı sağa aktarmaz.
  - İsteği olmayan cihaz acknowledge'ı sağa aktarır.
- Örnekler (Device 0, 1, 2 istekleri):
  - 1, 0, 0: Device 0'a hizmet verilir.
  - 1, 1, 0: yine Device 0. Device 0'ın PO çıkışı 0 olur.
  - 0, 1, 1: Device 0 acknowledge'ı aktarır. Device 1 hizmet alır, Device 2'ye aktarmaz. Device 2, Device 1'in isteği bitince hizmet alır.
- Bir bloğun iç yapısı: SR latch (RF), gecikme (delay), iki AND kapısı, vektör adres register'ı.

PI	RF	PO	Enable	Anlamı
0	0	0	0	Acknowledge yok, istek yok
0	1	0	0	İstek var ama daha öncelikli bir cihaza hizmet veriliyor
1	0	1	0	Acknowledge geldi, istek yok → sağa aktar
1	1	0	1	Acknowledge geldi, istek var → VAD'ı bırak

- Satırları derste bir öğrenci yorumladı.
- Daisy chain seri çalıştığı için biraz yavaştır.

6. Paralel öncelik donanımı

- 4 kaynağın istekleri interrupt register'ında tutulur. Bit 3 en yüksek, bit 0 en düşük önceliklidir.
- **Mask register:** ilgili bit 0 ise o kaynağın isteği AND kapısıyla engellenir. Yazılımla programlanır.
- **4'ten 2'ye öncelik kodlayıcı:**

D3 D2 D1 D0	A1 A0	V
0 0 0 0	X X	0
0 0 0 1	0 0	1
0 0 1 X	0 1	1
0 1 X X	1 0	1
1 X X X	1 1	1

- A1 ve A0 vektör adresinin (VAD) en düşük iki bitini oluşturur. Diğer bitler 0'dır.
- V çıkışı CPU'ya kesme isteği olarak gider.
- Bir öğrenci maskelenemeyen kesmeleri (NMI) sordu. Hoca: kritik kesmelerdir, hiçbir şekilde maskelenemez. Bu yapıdaki mask register ise yazılımla programlanır.

7. Doğrudan bellek erişimi (DMA)

- Amaç: bellek ile G/Ç arasındaki transferi hızlandırmak. CPU devre dışı bırakılır.
- **Bus request (BR) / bus grant (BG):**
  - DMA denetleyicisi BR gönderir.
  - CPU adres yolu, veri yolu ve read/write hatlarını yüksek empedansa (high-Z) çeker ve BG'yi 1 yapar.
  - Artık yolu DMA denetleyicisi yönetir.
- **DMA denetleyicisi** de bir arayüzdür:
  - CPU tarafı: DS (DMA select, chip select), RS (register select), RD, WR, BR, BG, interrupt.
  - G/Ç tarafı: DMA request, DMA acknowledge.
  - İç register'lar:
    - **Adres register'ı:** transferin bellekteki adresi. Her transferden sonra 1 artırılır.
    - **Word count register'ı:** transfer edilecek veri miktarı. Her transferden sonra 1 azaltılır.
    - **Control register:** transferi denetler.
  - Bazı arayüzlerde aynı adres sırayla okununca farklı register'lara erişilir.
- **Adımlar:**
  1. Cihaz DMA denetleyicisine DMA request gönderir.
  2. CPU denetleyicinin iç register'larını yükler (ör. `OUT 0A8h, AL` ile adres, `OUT 0A9h, AL` ile word count).
  3. Denetleyici BR gönderir.
  4. CPU yolları bırakır ve BG'yi 1 yapar.
  5. Denetleyici adresi adres yoluna koyar, okuma/yazma yönünü seçer ve cihaza DMA acknowledge gönderir.
  6. Veri cihazdan belleğe (ya da tersi) doğrudan veri yolu üzerinden aktarılır. Adres +1, word count −1. Bitene kadar devam eder.
- Her byte CPU register'ından geçmediği için çok daha hızlıdır.

8. Örnek problemler

- **Problem 1: Arayüzü 240-243 (onluk) adreslerine yerleştirme (8 bit G/Ç adresi)**
  - 240 = `1111 0000`, 243 = `1111 0011`. Değişen bitler: A1, A0 → RS1, RS0.
  - A7-A4 = 1111 → 4 girişli AND. A3, A2 = 00 → NOR.
  - İki kapının çıkışı bir AND kapısına girer → CS.
  - Veri yolu doğrudan arayüze bağlanır.
- **Problem 2: 16 bitlik adresle kaç arayüz bağlanabilir?**

  - **Doğrusal seçim (linear select):** adres çözücü yok. A1, A0 bütün arayüzlerin RS uçlarına bağlanır. Kalan 14 hattın her biri bir arayüzün CS ucuna bağlanır → en çok **14 arayüz**.
    * A15'e bağlı arayüz yalnızca diğer hatlar 0 iken seçilmelidir. Adresleri 8000h-8003h.
    * A14'e bağlı arayüzün adresleri 4000h-4003h.
    * Bu yöntem birçok adres bölgesini kullanılamaz hâle getirir.
  - **Tam adres çözme (fully decoded):** 14 bit bir çözücüden geçerse $2^{14} = 16384$ arayüz bağlanabilir.
- **Problem 3:** önceki yılların sınav sorusu, size bırakıldı.
- **Problem 4: Seri iletim süresi**
  - 57600 bit/s hızla 'A' karakteri (41h) asenkron, even parity ve 2 stop bitiyle gönderiliyor.
  - 41h = `0100 0001`. D0'dan başlanır: 1, 0, 0, 0, 0, 0, 1, 0.
  - Veride iki tane 1 var → even parity biti 0.
  - Toplam: 1 start + 8 veri + 1 parity + 2 stop = 12 bit.
  - Süre: $12 \times \frac{1}{57600} \approx 208.3\ \mu s$.

9. Önbelleğe giriş

- **SRAM ve DRAM:**
  - Önbellek SRAM ile yapılır. SRAM hücresi latch tabanlıdır, hızlıdır ama pahalıdır.
  - Ana bellek DRAM ile yapılır. DRAM kapasitör kullanır ve tazelenmeye (refresh) ihtiyaç duyar. Daha yavaş ve ucuzdur.
  - Maliyet yüzünden önbellek ana bellekten çok küçük tutulur.
- **Hit ve miss:**
  - CPU veriyi önce önbellekte arar. Bulursa hit, bulamazsa miss.
  - Miss olursa veri ana bellekten okunur.
- Önbellek komut önbelleği (instruction cache) ve veri önbelleği (data cache) olarak ayrı da olabilir.
- Üç eşleme (mapping) yöntemi vardır.

10. Direct mapped önbellek

- Örnek: 10 bitlik adres. Ana bellekte her satır 4 byte (32 bit). Bu yüzden satır adresleri 0, 4, 8, ... diye gider.
- Önbellek 8 satır → index $\log_2 8 = 3$ bit.
- Byte offset: $\log_2 4 = 2$ bit.
- Tag: $10 - 3 - 2 = 5$ bit.

Tag (5)	Index (3)	Byte offset (2)

- Örnek: ana bellekteki 4 adresinin index'i 001 → önbelleğin 001 satırına yazılır. Tag olarak 00000 da yazılır.
- Önbellek veriyle birlikte tag'i de tutar. Böylece o satırda hangi adresin verisi olduğu bilinir.
- **Dezavantaj:** index'i aynı olan adreslerden yalnızca biri önbellekte durabilir. Yenisi eskinin üzerine yazılır.

11. Fully associative önbellek

- Index yoktur. Adres: tag (8 bit) + byte offset (2 bit).
- Veri önbelleğe sırayla yazılır. Önbellek dolunca yer değiştirme algoritması gerekir:
  - LRU (least recently used): son zamanlarda en az kullanılanın üzerine yazılır.
  - Rastgele yer değiştirme (random replacement).
- **Dezavantaj:** verinin hangi satırda olduğu bilinmez. CPU'nun adresindeki tag, bütün satırların tag'iyle paralel karşılaştırılmalıdır.

  - Satır sayısı kadar eşleştirme devresi gerekir (ör. 1024 satır için 1024 devre). Maliyet artar.
- **Eşleştirme lojiği:** her tag biti çifti için XOR. Bitler eşitse XOR çıkışı 0'dır. Bütün XOR çıkışları NOR kapısına girer. NOR = 1 → eşleşme.

12. Set associative önbellek

- Direct mapped yapının çoğullanmış hâlidir.
- Örnek: toplam 8 satır, iki parça (set 0 ve set 1), her biri 4 satır → 2-way set associative.
  - Index: 2 bit. Byte offset: 2 bit. Tag: 6 bit.
- Aynı index'e sahip iki farklı adres aynı anda önbellekte tutulabilir. Tek avantajı budur.
- 4-way ve 8-way yapılar da kullanılır.
- **Kaç karşılaştırıcı gerekir?** Index ile satır bilinir. O satırdaki iki tag karşılaştırılır → 2 eşleştirme devresi. Çıkışları OR'lanır: 1 ise hit, 0 ise miss.

13. Blok (line) kavramı

- Word, işlemcinin veri yolu genişliğidir. 8086'da 16 bit, bu örnekte 32 bit (4 byte).
- Veri önbelleğe word yerine blok (line) olarak taşınabilir. Bir blok 2, 4 ya da 8 word olabilir. Bu daha verimlidir.
- Örnek (2-way set associative, 4 satır, blok = 4 word):
  - Byte offset 2 bit, word offset 2 bit, index 2 bit, tag $10 - 6 = 4$ bit.
- Blok örnekleri sonraki derste çoğaltılacak.

Ders sonrası notlar

- **RS-232 boşta gerilimi:** Hoca "hat boştayken +5 V ya da RS-232'de +12 V" dedi. Hat boşta iken lojik 1'dir. TTL'de bu +5 V'tur. RS-232'de ise lojik 1 (mark) negatif gerilimle gösterilir (-3 V ile -15 V arası, tipik -12 V). +12 V, RS-232'de lojik 0'dır.
- **Seri iletim örneği:** Hoca "stop bitiyle başladınız" ve "bir tane stop biti ekledik" dedi. Karakter **start** bitiyle başlar. Eklenen bitler: 1 start + 1 parity + 2 stop. Ayrıca 41h tahtada 9 haneli (`010000001`) yazıldı. Doğrusu 8 bittir: `01000001`.
- **Daisy chain tablosu, üçüncü satır:** Hoca "1, 0, 1 olsun ya da 0, 0, 1 olsun" dedi. $1, 0, 1$'de Device 0 hizmet alır ve Device 1'in PI girişi 0 olur. Device 1'de bu satırı yalnız $0, 0, 1$ oluşturur.
- **DMA register yükleme:** Hoca word count için "AL'yi ya da AH register'ını ayarlayıp" dedi. 8 bitlik `OUT` komutunun kaynağı AL olmalıdır. `OUT 0A9h, AH` GNU as ile denendi, reddedildi. Yüksek bayt önce AL'ye taşınır.
- **Parity:** Hoca "parity biti dahil bu 8 bitlik sayı" dedi. Parity biti dahil 9 bit sayılır.
- **Port C:** Doğrusal seçim örneğinde hoca "port C register'ı" dedi. Dersteki örnek arayüzün register'ları Port A, Port B, Control ve Status'tur. Port C yoktur.
- **Seri iletim süresi:** $12/57600 \approx 208.3\ \mu s$. Python ile doğrulandı.
- **Paralel öncelikte vektör adresi:** Hoca Device 3 için vektör adresini "0123 / 013 hex" diye okudu. Ekrandaki şekle göre VAD 8 bittir. A0 en düşük, A1 bir sonraki bittir, kalan 6 bit 0'dır. Bu yüzden Device 0-3 için VAD sırasıyla 00h, 01h, 02h, 03h olur.
- **Doğrusal seçim adresleri:** Hoca "80000, 800001…" ve "40001…" diye okudu. 16 bitlik adresler 4 hex hanelidir: 8000h-8003h ve 4000h-4003h.
- **Eşleştirme lojiği:** Hoca bir yerde "4 XOR ve bir OR" dedi. Hemen önce anlattığı gibi son kapı **NOR**'dur. Bütün XOR çıkışları 0 iken NOR çıkışı 1 olur.
- **"Set" ve "way" terimleri:** Kitaptaki şekil iki paralel bölüme "set 0" ve "set 1" diyor. Yaygın terminolojide aynı index'teki satırların grubuna **set**, paralel bölümlerin her birine **way** (yol) denir. Bu örnek 4 set, 2 way'dir. "Set sayısı kadar karşılaştırıcı" ifadesi bu yüzden "way sayısı kadar karşılaştırıcı" diye okunmalıdır.

- **Önbellek boyutu:** Hoca önbelleğin ana belleğin “onda biri, yirmide biri” kadar olduğunu söyledi. Günümüz bilgisayarlarında oran çok daha küçüktür: birkaç MB önbellek, birkaç GB ana bellek.
- **SRAM hücresi:** Hoca “SR latch” dedi. Tipik SRAM hücresi çapraz bağlı iki eviriciden oluşan bir latch’tir (6 transistör). Mantık olarak doğru.
- Altyazıda “Manon” Mano, “IORID” I/O Read/I/O Write, “Eknolci/Ekne oluş” acknowledge, “Baskrent” bus grant, “SET Asosyatif” set associative demektir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.